

Improving the Efficiency of Global EDF Exact Schedulability Test

Artem Burmyakov* and Enrico Bini†

* Innopolis University, Russia

†University of Turin, Italy

Abstract—Exact schedulability tests are a key element of the analysis of real-time systems as they check temporal constraints without sacrificing processing capacity. However, achieving both exactness and computational efficiency for the same test may be challenging. For instance, Global Earliest Deadline First (GEDF) on multi-cores has no known worst-case scenario for tasks releases, implying that an exact test must check all possible release scenarios. Consequently, existing GEDF exact tests can only check 4-5 tasks with tiny parameters executed upon 2 cores.

We propose an improved exact GEDF schedulability test for sporadic real-time tasks. Our test is many orders of magnitude faster and less memory consuming than any other available exact test for GEDF. Its C++ implementation is publicly available.

Index Terms—Real-time Multiprocessor Scheduling, Global Earliest Deadline First, Exact Schedulability Test.

I. INTRODUCTION

The design of real-time systems requires schedulability tests. These tests ensure that deadlines of tasks are met. The tests typically trade accuracy vs. efficiency. If tests must be performed at run-time, then efficiency is favored. On the other hand, a time-consuming test is affordable at design-time. It is hence preferable to have accurate tests because of a higher accuracy leading to a more efficient usage of the processing capacity.

Global Earliest Deadline First (GEDF) [1] is a widely used multicore scheduler. It is for example implemented in Linux by the `SCHED_DEADLINE` scheduling class. Until recently, an exact schedulability test for sporadic tasks scheduled by GEDF was an open problem [2]. Then, Bonifaci et al. proposed the first exact test, which is NP-hard [3].

The key reason for the hardness of the problem is that, unlike the single core case, no worst-case jobs release scenario for sporadic tasks is known for a multicore case. Due to the lack of the worst-case execution scenario, it is necessary to examine all possible release scenarios [3]. The quick growth of the search space with the number of tasks and their parameters makes the schedulability test prohibitive even for simple task systems comprised of 4–5 sporadic tasks with tiny parameters. Thus, we propose to further evolve their idea by employing a set of graph pruning techniques, which are presented in later sections.

A. Related works

The verification of real-time temporal constraints was addressed by the research community under many different angles. Alur and Dill [4] were among the first ones to propose

the usage of *timed automata*, an extension of automata with clocks measuring the time elapsing between events. Behrmann et al. [5] proposed the usage of Clock Difference Diagrams (CDDs) to reduce the memory consumption by the analysis, which was later adopted by the UPPAAL verification tool [6]. Audemard et al. [7] employed the bounded model checking approach combined with timed automata. In the context of actor-based programming languages, Khamespanah et al. [8] proposed Timed Rebeca verification approach by using a time shift operator for mitigating the states explosion problem. Henzinger et al. [9] applied the symbolic model checking approach to the verification of real-time properties, and Bouajjani et al. [10] made it “on-the-fly” by dynamically generating the state-space and hence storing only the minimal information in memory. Bini et al. [11] proposed to determine the region of schedulable task parameters by combining the set of inequalities from schedulability conditions, and Cimatti et al. [12] proposed a similar result for a task model with offsets by exploiting parametric timed automata. Pinxten et al. [13] adopted their approach for non-preemptive real-time scheduling for the case of the Symbolic Shortest Processing Time First (SPTF) scheduler. Abdeddaïm and Maler [14] used timed automata to model a traditional job-shop scheduling. Baker and Cirinei [15] were first to propose an exact schedulability test for several multicore schedulers by employing state-transition graphs, and Bonifaci and Marchetti-Spaccamela [3] then further elaborated their approach. Geeraerts et al. [16] instead addressed the problem by relying on formal verification methods. Goossens et al. [17] determined the exact simulation interval for schedules of periodic tasks over multiprocessors. Other works are dedicated to other schedulers and task models, such as by Sun and Lipari [18], Guan et al. [19], Goossens and Yomsi [20], and Nasri and Brandenburg [21]. Picard et al. [22] proposed a finite-state automaton approach to the problem of sporadic mixed-criticality tasks.

Recently the Schedule Abstraction Graphs (SAG) framework was proposed to verify schedulability of different task models [21], [23]. To the best of our knowledge, SAG is however inapplicable to a general *sporadic* task model, due to its following requirements:

- a tasks release scenario to be finite;
- the number of every task releases to be known;
- uncertain release times to be bounded.

These constraints reduce the complexity of an exact schedulability test. Instead, our approach does not employ none of

them, but rather uses state space pruning rules, which make the test more tractable while preserving its exactness. Our work is thus totally incomparable to the SAG framework.

The distinguishing feature of our work, compared to vast contributions in the area, is the exploitation of scheduler-specific knowledge yielding a faster exact test than the-best-in-class existing work [3]. A pruning-driven test was initially introduced for the Global Fixed Priority (GFP) scheduler [24]. However, as pruning rules are scheduler-dependent, they are either invalid or must be formally proved again for GEDF.

Contributions and paper organization: Our contributions are:

- 1) A GEDF exact schedulability test driven by state-space pruning, which outperforms other best-in-class tests;
- 2) A publicly available C++ test implementation at <https://github.com/burmyakov/exact-sched-test-gedf>;
- 3) Evaluation of scheduling performance for Bertogna's state-of-the-art sufficient test [25], [26] over our exact one.

The paper is organized as follows. In Section II, we formalize the model of tasks execution by means of a finite state transition graph. In Section III, we provide a high-level overview of our entire analysis. In Section IV we describe the pruning rule based on the concept of the worst-case states, and in Section V we illustrate an optimized choice for the visiting order of the graph states, to maximize the efficiency of the pruning rule from IV. Then in Section VI we describe another pruning rule exploring the requirement of jobs interference. Finally, in Section VII we report the evaluation results of the proposed exact schedulability test.

II. MODEL OF TASKS EXECUTION

A real-time system $\mathcal{T}$ is modeled by a set of n *sporadic tasks* $\{\tau_1, \dots, \tau_n\}$, where each task $\tau_i = (C_i, D_i, P_i)$ is modeled by the worst-case *execution time* C_i , a *relative deadline* D_i , and a *minimum interarrival time* P_i between the releases of consecutive jobs. Also, the quantity $U_i = C_i/P_i$ is the task *utilization* and $U_{\mathcal{T}} = \sum_{\tau_i \in \mathcal{T}} U_i$ is the utilization of the entire task set $\mathcal{T}$. Each task $\tau_i \in \mathcal{T}$ releases a potentially infinite sequence of *jobs* in accordance to the minimum interarrival time P_i : if τ_i releases its j -th job at time $r_{i,j}$, then it can release its next job at any time $r_{i,j+1}$, such that

$$r_{i,j+1} \geq r_{i,j} + P_i. \quad (1)$$

Any j -th job of τ_i released at $r_{i,j}$ requires at most C_i units of CPU time and must complete not later than the *absolute deadline* $r_{i,j} + D_i$, otherwise the job is said to *miss a deadline*.

We have m cores and we consider a GEDF scheduler [1] to schedule tasks on them. In GEDF, any job released at time $r_{i,j}$ is enqueued into a single queue in accordance to the deadline at $r_{i,j} + D_i$, that is D_i units after the job release. The m non-completed jobs with the earliest deadlines are then scheduled over any of the m cores. The migration of jobs among cores is allowed. We assume that scheduling decisions are taken at discrete times because “a discrete schedule exists whenever a continuous schedule does” [3]: task parameters C_i , D_i , P_i are integers, and task deadlines are constrained by $D_i \leq P_i$. At

the top right corner of Fig. 1, we provide the GEDF schedule of two possible release scenarios for the sporadic tasks $\tau_1 = (1, 2, 2)$, $\tau_2 = (2, 3, 3)$, and $\tau_3 = (3, 4, 4)$ scheduled upon $m = 2$ cores.

Continuing on the idea by Bonifaci and Marchetti-Spaccamela [3], we construct a graph with each node representing a possible scenario for job releases. An example of such a graph is depicted in Fig. 1, and below we formalize it. We use the *state* x defined by

$$x = [(c_1, p_1), \dots, (c_n, p_n)], \quad (2)$$

to represent the current state of the tasks in $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$, where

- $c_i \in \{0, \dots, C_i\}$ is the remaining execution time of a pending job of τ_i (if $c_i = 0$, then τ_i is not pending) and
- $p_i \in \{0, \dots, P_i\}$ is the earliest time until task τ_i can release the next job,

thus the whole *state space* $\mathcal{X}$ is a finite set of all possible states x defined by

$$\mathcal{X} = \prod_{i=1}^n \{0, \dots, C_i\} \times \{0, \dots, P_i\} \subset \mathbb{N}^{2n}. \quad (3)$$

As the system starts with no pending jobs, $\mathbf{0} = [(0, 0)]_{i=1}^n \in \mathcal{X}$ denotes the *initial state* with $c_i = 0$ (no task has a pending job) and $p_i = 0$ (every task is allowed to release a job at the next time tick).

For real-time systems, all task deadlines must be met. Hence, only some states are feasible, whereas states leading to a deadline miss are not. We define the *set of feasible states* $\mathcal{F}$ as

$$\mathcal{F} = \{[(c_i, p_i)] \in \mathcal{X} : c_i \leq d_i\} \quad (4)$$

with d_i denoting the time left until the current deadline of task τ_i computed from the state variable p_i by

$$d_i = \max(p_i - P_i + D_i, 0). \quad (5)$$

The expression (5) computes the time left until the deadline of a currently pending job for τ_i , if any, because: p_i is the time to its next possible release, $p_i - P_i$ is the release time of a currently pending job for τ_i , and, consequently, $p_i - P_i + D_i$ is the remaining time left until its deadline. The inequality in (4) is then encoding that the remaining execution time c_i cannot exceed the remaining time until the deadline d_i , otherwise τ_i will certainly miss its deadline. We remark that the initial state is feasible, $\mathbf{0} \in \mathcal{F}$.

In one time unit, the transition from a given state x at time t to its *neighbor successor* x^+ at time $t + 1$ is affected by:

- the scheduler selecting jobs for execution over the interval $[t, t + 1)$, and
- tasks releasing new jobs at time $t + 1$.

We formalize these notions next.

At any state $x = [(c_i, p_i)]_{i=1}^n$, the set $\mathcal{Q}(x) \subseteq \mathcal{T}$ denotes the *pending tasks* formally defined by

$$\mathcal{Q}(x) = \{\tau_i \in \mathcal{T} : c_i > 0\}. \quad (6)$$

In words, a task is pending (that is $\tau_i \in \mathcal{Q}(x)$) when it has some work to be completed ($c_i > 0$).

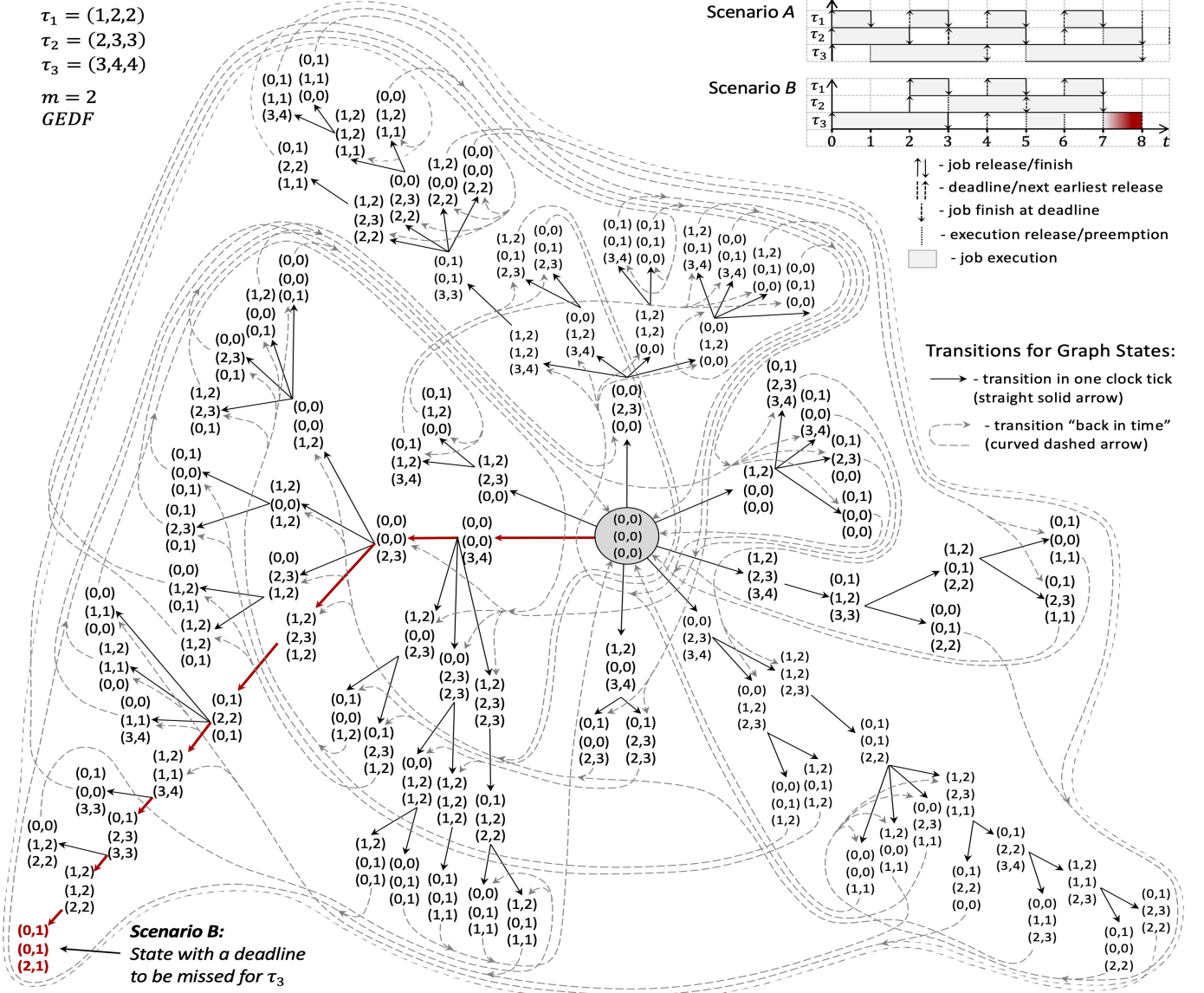


Fig. 1: The full state transition graph for tasks $\tau_1 = (1, 2, 2)$, $\tau_2 = (2, 3, 3)$, $\tau_3 = (3, 4, 4)$ scheduled upon 2 cores by GEDF. States are represented by the tuples (c_1, p_1) , (c_2, p_2) , and (c_3, p_3) , vertically stacked. The schedule of the release Scenario B is represented at the top right corner and it leads to a deadline violation. Such a scenario corresponds to the state trajectory $\vec{x}$, depicted by the path of thick arrows. The deadline violation is detected as such a trajectory leads to the infeasible state $[(0, 1), (0, 1), (2, 1)] \notin \mathcal{F}$, with $\mathcal{F}$ defined in (4). Such a state is not feasible because $c_3 = 2 > d_3 = 1$, meaning that the remaining execution time is larger than the time to the deadline. We remark that the release scenario corresponding to a deadline miss is **not** when jobs are released as soon as possible.

The *scheduler* is a function $\mathcal{S} : \mathcal{X} \rightarrow 2^{\mathcal{T}}$, with $\mathcal{S}(x) \subseteq \mathcal{Q}(x)$, indicating the subset of tasks being scheduled over the next unit interval, when the state is x . Specifically for GEDF scheduler, if the number of pending jobs does not exceed the number of cores m , then all these jobs are scheduled, otherwise only m jobs with the earliest deadlines are scheduled. To formalize the GEDF definition we first define, for a given state $x = [(c_i, p_i)]_{i=1}^n$ and a task τ_i , the set of *higher priority* tasks as

$$\mathcal{T}^{\text{HP}}(x, \tau_i) = \{ \tau_j \in \mathcal{T} : c_j > 0 \wedge (d_j < d_i \vee (d_j = d_i \wedge j < i)) \}, \quad (7)$$

meaning that τ_j has a priority higher than τ_i if:

- 1) τ_j has a pending job (that is $c_j > 0$) and
- 2) the next τ_j deadline ($d_j = p_j - P_j + D_j$ from (5)) is:

- a) earlier than the deadline d_i of τ_i , or
- b) τ_j has a lower index than τ_i , in case of same deadlines d_j and d_i .

The GEDF scheduler is then formalized as follows.

Definition 1 (GEDF scheduler): Given a state $x = [(c_i, p_i)]_{i=1}^n$ and a set $\mathcal{Q}(x)$ of pending tasks defined by (6), the *GEDF scheduler* $\mathcal{S} : \mathcal{X} \rightarrow \mathcal{T}$ is defined by:

- if $|\mathcal{Q}(x)| \leq m$, then $\mathcal{S}(x) = \mathcal{Q}(x)$;
- otherwise, $\mathcal{S}(x) \subset \mathcal{Q}(x)$ such that $|\mathcal{S}(x)| = m$ and

$$\forall \tau_j \in \mathcal{Q}(x), \tau_j \notin \mathcal{S}(x), \quad \mathcal{S}(x) \subseteq \mathcal{T}^{\text{HP}}(x, \tau_j).$$

The definition of possible job releases follows from the constraint (1) of minimum separation between two consecutive job release instants of the same sporadic task τ_i . According to (1), for a given state $x = [(c_i, p_i)]_{i=1}^n \in \mathcal{X}$, the job releases

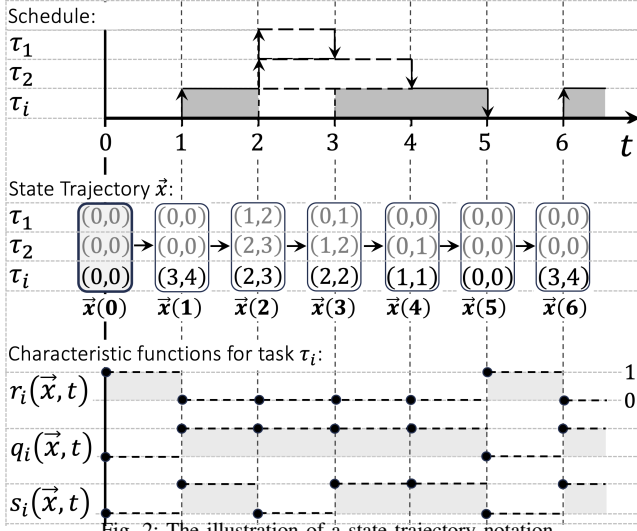


Fig. 2: The illustration of a state trajectory notation

denoted by $r = [r_1, \dots, r_n]$ must belong to the *legal releases* $\mathcal{R}(x) \subseteq \{0, 1\}^n$ defined by

$$\mathcal{R}(x) = \{[r_i] \in \{0, 1\}^n : p_i > 1 \Rightarrow r_i = 0\}, \quad (8)$$

with $r_i = 1$ indicating a new τ_i job release at the next time, and $r_i = 0$ otherwise. The reason for the “ $p_i > 1$ ” inequality in (8) is that task τ_i may release a new job in one time instant from any state x having $p_i \leq 1$.

The *state transition law* $\Phi(x, r) \in \mathcal{X}$ determines the neighbor successor $x^+ = [(c_i^+, p_i^+)]$ from a given state x and with some legal releases $[r_i] \in \mathcal{R}(x)$ by

$$\Phi([(c_i, p_i)], [r_i]) = [(c_i^+, p_i^+)],$$

$$\text{with } \begin{cases} c_i^+ = c_i - 1 + r_i C_i, & i \in \mathcal{S}(x) \\ c_i^+ = c_i + r_i C_i, & i \notin \mathcal{S}(x) \\ p_i^+ = \max(p_i - 1, 0) + r_i P_i. \end{cases} \quad (9)$$

That is, as illustrated in Fig. 2, for any transition from a given state x , the following holds, for every τ_i :

- if τ_i is scheduled at state x (that is $i \in \mathcal{S}(x)$), the pending execution time c_i^+ at its successor x^+ is decremented by one ($c_i^+ = c_i - 1$);
- if the task τ_i is released at state x^+ (that is $r_i = 1$), the pending execution time c_i^+ is incremented by the task execution time C_i ;
- the remaining time until the next possible release p_i^+ is decreased by one until reaching zero, independently of either a job of τ_i is scheduled at x , or incremented by the task period P_i in case of a new τ_i job release at state x^+ (that is $p_i^+ = \max(p_i - 1, 0) + r_i P_i$).

Upon the releases and execution of jobs, the state of the system follows a trajectory, formally defined next.

Definition 2 (Trajectory $\vec{x}$): A state trajectory $\vec{x}$ is a sequence of states over time $\vec{x} : \mathbb{N} \rightarrow \mathcal{X}$ with

$$\vec{x}(0) = \mathbf{0} \quad (10)$$

$$\forall t \in \mathbb{N}, \exists r \in \mathcal{R}(\vec{x}(t)), \quad \vec{x}(t+1) = \Phi(\vec{x}(t), r). \quad (11)$$

Eq. (10) requires that at the initial state, there is no pending work ($c_i = 0$) and all tasks may release a job ($p_i = 0$). Eq. (11)

ensures that the next state in the trajectory always corresponds to legal releases, as defined in Eq. (8). Hence, the trajectory $\vec{x} = [\vec{x}(t)]_{t \in \mathbb{N}} = [\vec{x}(0), \vec{x}(1), \vec{x}(2), \dots]$ represents the system evolution over time and $x(\mathbb{N}) \subseteq \mathcal{X}$ is the set of all states visited by the trajectory $\vec{x}$.

For a given trajectory $\vec{x} = [(c_i(t), p_i(t))]_{t \in \mathbb{N}}$, it is convenient to define the *pending jobs sequence* and the *task schedule* respectively by

$$q_i(\vec{x}, t) = \begin{cases} 1 & \text{if } i \in \mathcal{Q}(\vec{x}(t)) \\ 0 & \text{otherwise} \end{cases}, \quad s_i(\vec{x}, t) = \begin{cases} 1 & \text{if } i \in \mathcal{S}(\vec{x}(t)) \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

which are indicator functions for $\mathcal{Q}(\vec{x}(t))$ and $\mathcal{S}(\vec{x}(t))$, respectively, as shown in Fig. 2.

We are now ready to define the schedulability of $\mathcal{T}$.

Definition 3 (Schedulability of $\mathcal{T}$): Given a GEDF scheduler $\mathcal{S}$ defined by Def. 1 and trajectories defined by Def. 2, the task set $\mathcal{T}$ is GEDF-schedulable if $\forall t \in \mathbb{N}, \vec{x}(t) \in \mathcal{F}$.

III. THE ANALYSIS IN BRIEF

After formalizing the task execution model for the GEDF scheduler, next in Algorithm 1 we outline the overall procedure of the proposed schedulability test.

The algorithm maintains two data structures:

- the set $\mathcal{V}$ of explicitly visited states, initialized with the empty set, and
- a list $\mathcal{L}$ of states to be examined at the next iterations.

From the Def. 3 of schedulability, we need to check all trajectories $\vec{x}$ originating from the initial state $\vec{x}(0) = \mathbf{0}$ for the presence of non-feasible states x (that is some $x \notin \mathcal{F}$ of (4) is found). This is made as follows:

- 1) The list $\mathcal{L}$ is initialized with the only initial state $\mathbf{0} = [(0, 0)]_{i=1}^n \in \mathcal{F}$;
- 2) At every iteration, a state x to be explored is fetched from $\mathcal{L}$ at Line 5, and then all its possible successors are generated by Eq. (9) and stored into $\mathcal{L}^+$.
- 3) Our key optimizations are at Line 7. Successors $\mathcal{L}^+$ of x are pruned by $\text{PRUNENEXTSTATES}(\mathcal{L}^+, \mathcal{V})$ considering already visited states $\mathcal{V}$. The details are in Sections IV and VI.
- 4) Each state $x^+ \in \mathcal{L}^+$ is checked; if x^+ is non-feasible, task set $\mathcal{T}$ is unschedulable.
- 5) Otherwise, x^+ is added to the visited states $\mathcal{V}$ and stored into $\mathcal{L}$ among the states to be explored next.

Algorithm 1 is guaranteed to terminate due to a finite number of possible states [3]. Our key contributions are the GEDF-specific pruning rules in the function $\text{PRUNENEXTSTATES}(\mathcal{L}^+, \mathcal{V})$, which reduce the runtime and memory consumption by several orders of magnitude. For the example of Fig. 1, the number of possible states upper bounded by Eq. (2) is 1440. However, the number of states, if considering only legal trajectories, is 91. Our Algorithm can further reduce the visited states to 33.

Section IV, then, describes our first pruning rule. The second significant contribution to the reduction of the runtime and memory consumption is due to the order of the states in $\mathcal{L}$, as described in Section V.

Algorithm 1 Our pruning-based exact schedulability test

```

1: function EXACTSCHEDULABILITYTEST
2:    $\mathcal{V} \leftarrow \emptyset$  ▷ initialize  $\mathcal{V}$ 
3:    $\mathcal{L} \leftarrow \{0\}$  ▷ initialize  $\mathcal{L}$ 
4:   while  $\mathcal{L} \neq \emptyset$  do ▷ loop until states need to be visited
5:      $x \leftarrow \text{FETCH}(\mathcal{L})$ 
6:      $\mathcal{L}^+ \leftarrow \{x^+ \in \mathcal{X} : x^+ = \Phi(x, r), r \in \mathcal{R}(x)\}$  ▷
        $\mathcal{R}(x)$  from (8),  $\Phi$  from (9)
7:      $\mathcal{L}^+ \leftarrow \text{PRUNENEXTSTATES}(\mathcal{L}^+, \mathcal{V})$  ▷ see Sec. IV
8:     for each  $x^+ \in \mathcal{L}^+$  do
9:       if  $x^+ \notin \mathcal{F}$  then ▷ deadline miss
10:        return Unschedulable
11:       end if
12:        $\mathcal{V} \leftarrow \mathcal{V} \cup \{x^+\}$ 
13:        $\mathcal{L} \leftarrow \text{INSERT}(\mathcal{L}, x^+)$ 
14:     end for
15:   end while
16:   return Schedulable
17: end function

```

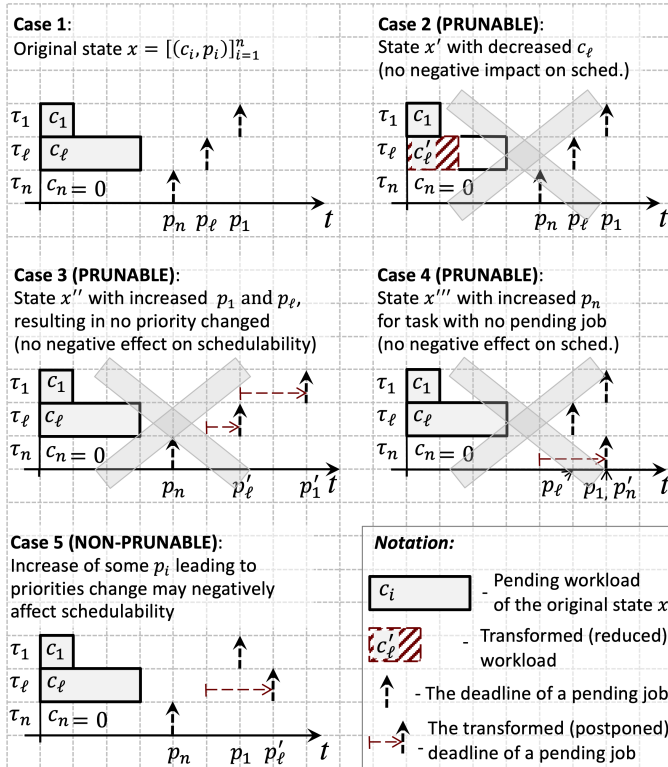


Fig. 3: Graphical illustration for Theorem 1: If no successor of state x misses a deadline, then no successor of states x' , x'' , and x''' misses a deadline. However, the same does not hold for the state in Case 5.

IV. WORST-CASE STATES

The first proposed pruning rule uses the concept of “worst-case” states. The idea in brief is to detect a “worst-case” state x such that if none of its successors misses a deadline, then no deadline may be missed from other states x' with less stringent “demand”. We first introduced this concept for the GFP case [24], but these results however are invalid for GEDF, due to its dynamically changing priorities.

Fig. 3 provides the intuition. For a given state x , hatched areas correspond to the values of c_i (pending work of τ_i),

dashed upward arrows represent the separations p_i to the earliest next release, and the deadlines d_i are implicitly computed by (5). Let us transform an original state $x = [(c_i, p_i)]_{i=1}^n$ corresponding to Case 1 of Fig. 3 into state x' of Case 2 by only decreasing the pending workload c_ℓ of τ_ℓ . Intuitively, such a workload reduction cannot negatively affect the schedulability of τ_ℓ or any other task. The same holds for Case 3 with some p_i (and consequently, deadlines) being increased, under the strong requirement of no job priority being changed. If, instead, the increase of p_i varies job priorities, as in Case 5, the schedulability of some task may be negatively affected. Finally, any increase of some p_n when $\tau_n \notin \mathcal{Q}(x)$ (that is τ_n is not pending in state x), as depicted in Case 4, can not negatively affect the schedulability. The next theorem formalizes this observation.

Theorem 1 (Worst-case states): Given task set $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$, let $\vec{x}$ be a trajectory generated by the releases $[r(t)]_{t \in \mathbb{N}}$. Let $\vec{x}'$ be a trajectory such that:

- 1) There exist two instants $\bar{t}$ and $\bar{t}'$ with states $\vec{x}(\bar{t}) = x = [(c_i, p_i)]_{i=1}^n$ and $\vec{x}'(\bar{t}') = x' = [(c'_i, p'_i)]_{i=1}^n$ such that

$$\begin{cases} c_i \geq c'_i \\ p_i \leq p'_i \\ \mathcal{T}^{\text{HP}}(x, \tau_i) \supseteq \mathcal{T}^{\text{HP}}(x', \tau_i) \end{cases}, \quad i = 1, \dots, n. \quad (13)$$

- 2) From instant $\bar{t}$, the releases $[r(t)]_{t \in \mathbb{N}}$ of $\vec{x}$ coincide with r' from $\bar{t}'$, that is:

$$\forall t \geq 0, \quad r(\bar{t} + t) = r'(\bar{t}' + t) \quad (14)$$

- 3) No deadline was missed in $\vec{x}$ until $\bar{t}$, and $\vec{x}'$ until $\bar{t}'$, that is $\forall t \leq \bar{t}, \vec{x}(t) \in \mathcal{F}$, and $\forall t \leq \bar{t}', \vec{x}'(t) \in \mathcal{F}$.

Then

$$\forall t > \bar{t}, \vec{x}(t) \in \mathcal{F} \Rightarrow \forall t > \bar{t}', \vec{x}'(t) \in \mathcal{F}. \quad (15)$$

In words, the statement of (15) means that if no state violates any deadline in trajectory $\vec{x}$, then no state violates any deadline in $\vec{x}'$. Hence, the trajectory $\vec{x}'$ can be safely pruned from the exploration, starting from the state x' . We remark that we are not determining the worst-case releases, which remain an open problem, to best of our knowledge. Rather, we reduce the number of release scenarios to be tested by comparing two state trajectories $\vec{x}$ and $\vec{x}'$.

Next lemma enables the proof of Theorem 1. In the lemma statement and in the proof of all the results, we use the notation “prime” as in $\vec{x}'$ and x' to denote quantities related to states that are going to be pruned.

Lemma 1: Let $\vec{x}$ and $\vec{x}'$ satisfy the hypothesis of Theorem 1. Then, it holds:

$$\forall i, t \geq 0 :$$

$$q_i(\vec{x}', \bar{t}' + t) = 1 \Rightarrow s_i(\vec{x}, \bar{t} + t) \leq s_i(\vec{x}', \bar{t}' + t), \quad (16)$$

where $q_i(\vec{x}, t)$ is the pending jobs sequence and $s_i(\vec{x}, t)$ is the schedule function along the trajectory $\vec{x}$ at t , as defined by (12).

According to the lemma above, if a task τ_i is scheduled at any time $\bar{t} + t$ in trajectory $\vec{x}$, then

- task τ_i is scheduled at time $\bar{t}' + t$ in trajectory $\vec{x}'$, or

- task τ_i does not have a pending job at time $\bar{t}' + t$ in $\bar{x}'$. Later, we will exploit this observation to prove Theorem 1: since no job in trajectory $\bar{x}'$ for instants $t > \bar{t}'$ can get less processing time than in $\bar{x}$ for $t > \bar{t}$ (due to the condition $s_i(\bar{x}', \bar{t}' + t) \geq s_i(\bar{x}, \bar{t} + t)$), then the trajectory $\bar{x}'$ starting from state $x' = \bar{x}'(\bar{t}')$ can be safely excluded from the analysis.

The proof of Lemma 1: First, we observe that for any given trajectory $\bar{x}'$ with its corresponding releases r' , it is always possible to build a legal trajectory $\bar{x}$ with releases r satisfying (14).

We prove the lemma by contradiction. Suppose that (16) is first violated for some task τ_{i^*} and t^* , that is

$$\exists i^*, t^* \geq 0 : \begin{cases} q_{i^*}(\bar{x}', \bar{t}' + t^*) = 1 \\ s_{i^*}(\bar{x}, \bar{t} + t^*) > s_{i^*}(\bar{x}', \bar{t}' + t^*), \end{cases} \quad (17)$$

$$\forall i, t \in [0, t^* - 1] : \quad q_i(\bar{x}', \bar{t}' + t) = 1 \Rightarrow s_i(\bar{x}, \bar{t} + t) \leq s_i(\bar{x}', \bar{t}' + t). \quad (18)$$

Note that, the hypothesis of (13) and the definition $q_{i^*}(\bar{x}, t)$ of τ_{i^*} 's pending jobs from (12) imply that it cannot be $t^* = 0$ for (17) to be true. Hence, $t^* > 0$ for sure.

Due to definition (12) of $s_i(\bar{x}, t)$, the assumption of (17) for t^* yields that:

1) from the strict inequality between binary functions:

$$s_{i^*}(\bar{x}, \bar{t} + t^*) = 1 \quad (19)$$

$$s_{i^*}(\bar{x}', \bar{t}' + t^*) = 0 \quad (20)$$

2) as τ_{i^*} is scheduled at $\bar{t} + t^*$ in $\bar{x}$, it must be pending:

$$q_{i^*}(\bar{x}, \bar{t} + t^*) = 1 \quad (21)$$

3) from hypothesis on t^* of (17):

$$q_{i^*}(\bar{x}', \bar{t}' + t^*) = 1 \quad (22)$$

From (20) and (22), task τ_{i^*} is pending but not scheduled in trajectory $\bar{x}'$ at time $\bar{t}' + t^*$. Hence, all m cores are busy with higher priority tasks, that is $|S(\bar{x}'(\bar{t}' + t^*))| = m$. In trajectory $\bar{x}$, instead, τ_{i^*} is scheduled, according to (19), so that there are at most $m - 1$ higher priority tasks at $\bar{t} + t^*$, that is $|\mathcal{T}^{\text{HP}}(\bar{x}(\bar{t} + t^*), \tau_{i^*})| \leq m - 1$. This means that there certainly exists a task $\tau_{i^{**}}$ such that:

1) $\tau_{i^{**}}$ is scheduled at state $\bar{x}'(\bar{t}' + t^*)$:

$$\tau_{i^{**}} \in \mathcal{S}(\bar{x}'(\bar{t}' + t^*)) \quad (23)$$

2) $\tau_{i^{**}}$ has either a lower priority job than τ_{i^*} , or does not have a pending job at all, at state $\bar{x}(\bar{t} + t^*)$:

$$\tau_{i^{**}} \notin \mathcal{T}^{\text{HP}}(\bar{x}(\bar{t} + t^*), \tau_{i^*}) \quad (24)$$

We denote the job of $\tau_{i^{**}}$ scheduled at $\bar{x}'(\bar{t}' + t^*)$ by $J'_{i^{**}}$.

We proceed by distinguishing the following possible cases depending on the status of job $J'_{i^{**}}$ in the trajectory $\bar{x}'$ and the status of $\tau_{i^{**}}$ in the trajectory $\bar{x}$.

- **[Case 1]** Task $\tau_{i^{**}}$ has no pending job at state $\bar{x}(\bar{t} + t^*)$:

$$q_{i^{**}}(\bar{x}, \bar{t} + t^*) = 0. \quad (25)$$

We further split this case depending on the relation between the release instant t'_r of the job $J'_{i^{**}}$ and the time instant $\bar{t}'$ defined in Theorem 1:

- **[Case 1(a)]** $J'_{i^{**}}$ is released at time $t'_r \leq \bar{t}'$, or
- **[Case 1(b)]** $J'_{i^{**}}$ is released at time $t'_r > \bar{t}'$.

- **[Case 2]** Task $\tau_{i^{**}}$ has a pending job $J_{i^{**}}$ at state $\bar{x}(\bar{t} + t^*)$:

$$q_{i^{**}}(\bar{x}, \bar{t} + t^*) = 1 \quad (26)$$

We next prove that all these cases contradict our assumptions (17)–(18).

Case 1(a): In the trajectory $\bar{x}'$, the job $J'_{i^{**}}$ is released at $t'_r \leq \bar{t}'$ and scheduled at $\bar{t}' + t^*$, from (23). Thus, the job $J'_{i^{**}}$ is always pending over $[\bar{t}', \bar{t}' + t^*]$. By evaluating the amount of time the job $J'_{i^{**}}$ is scheduled over $[\bar{t}', \bar{t}' + t^*]$, as well as considering (13), we find that:

1) From (23), as $\tau_{i^{**}}$ is not completed by $\bar{t}' + t^*$ in $\bar{x}'$:

$$\sum_{t=\bar{t}'}^{\bar{t}'+t^*-1} s_{i^{**}}(\bar{x}', t) < c'_{i^{**}},$$

2) From (25), as $\tau_{i^{**}}$ is finished by state $x(\bar{t} + t^*)$:

$$c_{i^{**}} = \sum_{t=\bar{t}}^{\bar{t}+t^*-1} s_{i^{**}}(\bar{x}, t)$$

3) From the theorem and lemma hypothesis of (13):

$$c'_{i^{**}} < c_{i^{**}},$$

what all together yield

$$\sum_{t=\bar{t}'}^{\bar{t}'+t^*-1} s_{i^{**}}(\bar{x}', t) < \sum_{t=\bar{t}}^{\bar{t}+t^*-1} s_{i^{**}}(\bar{x}, t),$$

meaning that there exists $t \in [0, t^* - 1]$ such that $s_{i^{**}}(\bar{x}, \bar{t} + t) > s_{i^{**}}(\bar{x}', \bar{t}' + t)$. For such a time t , it must be $q_{i^{**}}(\bar{x}', \bar{t}' + t) = 1$ because $\tau_{i^{**}}$ is always pending in trajectory $\bar{x}'$ over $t \in [0, t^* - 1]$. This contradicts (18) and makes **Case 1(a)** infeasible.

Case 1(b): In trajectory $\bar{x}'$, job $J'_{i^{**}}$ is released at $t'_r > \bar{t}'$ and is scheduled at $\bar{t}' + t^*$. From the property of the releases of (14), $\tau_{i^{**}}$ releases a job at $t_r = \bar{t} + (t'_r - \bar{t}')$ and such a job is finished at $\bar{t} + t^*$ because of (25). This implies

$$\sum_{t=t_r-\bar{t}}^{t^*-1} s_{i^{**}}(\bar{x}, \bar{t} + t) = C_i \quad \wedge \quad \sum_{t=t'_r-\bar{t}'}^{t^*-1} s_{i^{**}}(\bar{x}', \bar{t}' + t) < C_i,$$

which contradicts (18). Thus, **Case 1(b)** is also infeasible.

Case 2: From (23), job $J'_{i^{**}}$ is pending in state $\bar{x}'(\bar{t}' + t^*)$. Also, from (26), $\tau_{i^{**}}$ has a pending job in state $\bar{x}(\bar{t} + t^*)$, which we denote by $J_{i^{**}}$. This job $J_{i^{**}}$ has a lower priority than $J'_{i^{**}}$, relative to the job of task τ_{i^*} pending in both $\bar{x}(\bar{t} + t)$ and $\bar{x}'(\bar{t}' + t)$, due to (23) and (24). Considering the definition (13) of the worst-case states, some other task $\tau_{i^{***}}$ must release a job - let us denote it by $J'_{i^{***}}$ - at some time t^{***} , such that $\bar{t}' < t^{***} \leq \bar{t}' + t^*$, in trajectory $\bar{x}'$. Following the same reasoning as for **Case 1(b)** proof above, task $\tau_{i^{***}}$ also releases a job $J_{i^{***}}$ at time t^{***} in trajectory $\bar{x}$, which, unlike job $J'_{i^{***}}$ in trajectory $\bar{x}'$, completes its execution by time t^* . This fact yields the contradiction of assumption (18). Thus, **Case 2** is also infeasible, concluding the proof of the lemma. ■

We are now ready to prove Theorem 1.

Proof of Theorem 1: We prove the theorem by contradiction and show that the existence of some non-feasible state $\vec{x}'(t^{\text{dm}}) \notin \mathcal{F}$ for some instant $t^{\text{dm}} > \bar{t}'$ is not compatible with the premise (15) of Theorem 1. For this, let J'_i be the job of some task τ_i missing a deadline at t^{dm} , that is $c'_i(t^{\text{dm}}) > d'_i(t^{\text{dm}})$, with $d_i(t)$ denoting the remaining time to the deadline according to (5). Let t'_r be the release of J'_i . We consider two cases for t'_r : **[Case 1]** $t'_r \leq \bar{t}'$, and **[Case 2]** $t'_r > \bar{t}'$.

Case 1: The execution demand of job J'_i at time $\bar{t}'$ is $c'_i > 0$. We have:

- 1) From (15), $\vec{x}(t) \in \mathcal{F}$ implies that τ_i misses no deadline in $\vec{x}$:

$$c_i \leq \sum_{t=0}^{t^{\text{dm}}-1-\bar{t}} s_i(\vec{x}, \bar{t} + t)$$

- 2) Due to the assumption of violated (15) above, not enough time is scheduled for J'_i in $\vec{x}'$, which is released at $t'_r \leq \bar{t}'$, pending at time $\bar{t}'$, and to be completed by time t^{dm} :

$$\sum_{t=0}^{t^{\text{dm}}-1-\bar{t}'} s_i(\vec{x}', \bar{t}' + t) < c'_i$$

- 3) The hypothesis of (13):

$$c'_i \leq c_i,$$

what all together violates Lemma 1. Hence, the statement (15) must be true in this case.

Case 2: We denote by J_i the τ_i job released at $t_r = t'_r + \bar{t} - \bar{t}'$. Such a job exists because of the hypothesis on the releases of (14). Considering that, unlike job J'_i , job J_i meets its deadline at time t^{dm} , we contradict Lemma 1 as

$$\sum_{t=0}^{t^{\text{dm}}-t'_r-1} s_i(\vec{x}', t'_r + t) < C_i \quad \wedge \quad \sum_{t=0}^{t^{\text{dm}}-t_r-1} s_i(\vec{x}, t_r + t) = C_i,$$

requires that $s_i(\vec{x}, \bar{t} + t) > s_i(\vec{x}', \bar{t}' + t)$ for some $t \geq 0$, and for such a t task τ_i is pending. Hence, this case is infeasible too. The theorem is then proved. ■

Thanks to Theorem 1, in Algorithm 1, function $\text{PRUNENEXTSTATES}(\mathcal{L}^+, \mathcal{V})$, we can safely prune all states x' satisfying the next condition:

Pruning rule 1 (Dominated states pruning):

$$\forall x' \in \mathcal{L}^+,$$

$$\text{IF } \exists x \in \mathcal{V} : \forall i = 1, \dots, n$$

$$c_i \geq c'_i \wedge p_i \leq p'_i \wedge \mathcal{T}^{\text{HP}}(x, \tau_i) \subseteq \mathcal{T}^{\text{HP}}(x', \tau_i),$$

$$\text{THEN } \mathcal{L}^+ := \mathcal{L}^+ \setminus \{x'\}$$

(27)

For example, the application of pruning rule 1 to the tasks set of Fig. 1 reduces the examined number of states by around 30%. In general, however, such a reduction in the number of states drastically increases for larger task sets, reaching multiple orders of magnitude (see details in Section VII).

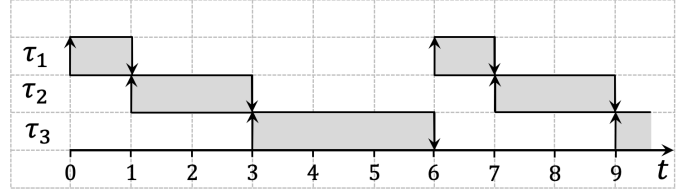


Fig. 4: Illustrating the intuition of the job interference condition for the same task set of Fig. 1. This scenario can be excluded from the analysis because no job interferes with any other job.

V. VISITING ORDER

The efficiency of the pruning rule (1) is higher if the worst-case states are detected sooner than other states. We have in Algorithm 1:

- Some state x' is pruned by PRUNENEXTSTATES function at Line 7, if there is some state x compliant to the Pruning rule 1 was already stored (x belongs to $\mathcal{V}$).
- If instead x' is visited before such x , then the Pruning rule 1 does not apply.

Hence, the order of states in list $\mathcal{L}$ to be efficient should maximize the chances of pruning by the Pruning rule 1. We have empirically established that such an efficient order is by an increasing *slack* $\text{slack}(x)$ defined as

$$\text{slack}(x) = \sum_{i=1}^n \text{slack}_i(x), \quad \text{slack}_i(x) = \begin{cases} 1 - \frac{c_i}{p_i}, & \text{if } c_i > 0 \\ 1, & \text{if } c_i = 0. \end{cases}$$

Intuitively, any state x with small $\text{slack}(x)$ still requires much processing time at future instants. Hence, if such a state x is feasible, it is more likely to enable the pruning of any other state x' through the Pruning rule 1. The proposed refinement of the visiting order gives a significant runtime reduction for Algorithm 1 compared to the original Bonifaci's order based on a simple comparison of c_i and p_i values. The detailed evaluation results are reported in Section VII.

VI. JOB INTERFERENCE REQUIREMENT

The second pruning rule exploits the requirement of every job to interfere with some other job. This rule was initially proposed for GFP schedulers [24]. We are now proving that it is applicable to GEDF as well.

Fig. 4 illustrates a fragment of schedule in which no interference happens. Such a scenario can then be pruned. We now generalize this observation and we show that we can safely prune any state, at which some job completes without interfering on the execution of any other job. In fact, such a state is dominated by other states, as demonstrated next.

We start with a formal definition of job interference.

Definition 4 (Job interference): Given a GEDF trajectory $\vec{x}$, let J_i be a τ_i 's job released at time t_r . We say that the job J_i *interferes* with some other job J_ℓ of τ_ℓ (with $\ell \neq i$) if

$$\exists t \in [t_r, t_r + D_i) :$$

$$\tau_i \in \mathcal{S}(\vec{x}(t)) \wedge \tau_\ell \in \mathcal{Q}(\vec{x}(t)) \wedge \tau_\ell \notin \mathcal{S}(\vec{x}(t)). \quad (28)$$

In words, Definition 4 states that J_i interferes with J_ℓ when the execution of J_i (encoded by $\tau_i \in \mathcal{S}(\vec{x}(t))$) prevents the execution of J_ℓ for some t .

To account for a job interference, we need to augment the definition of a system state x of Eq. (2) as follows

$$x = [(c_1, p_1, h_1), \dots, (c_n, p_n, h_n)], \quad (29)$$

where h_i is an added boolean variable indicating if a current job of τ_i has interfered with any other job by state x .

Hence, for the state x at time instant t , the variable h_i is governed by the following dynamics:

$$h_i(0) = 0, \\ h_i(t+1) = \begin{cases} 0 & \text{if } c_i = 0 \\ 1 & \text{if } \tau_i \in \mathcal{S}(x) \wedge |\mathcal{Q}(x)| \geq m+1 \\ h_i(t) & \text{otherwise,} \end{cases} \quad (30)$$

meaning that:

- if τ_i has no pending job at x , then $h_i(t+1) = 0$ for the succeeding states of x at time $t+1$;
- if instead τ_i is scheduled at x (that is $\tau_i \in \mathcal{S}(x)$) while some other τ_ℓ is pending but not scheduled (due to $|\mathcal{Q}(x)| \geq m+1$), then $h_i(t+1) = 1$, as from the definition (4) τ_i interferes with τ_ℓ at x .

This extra variable h_i is exploited in the following pruning rule. Reusing the notation of Alg. 1 at Line 7, the pruning rule takes the list $\mathcal{L}^+$ of states pending to be explored and erases $\forall x' \in \mathcal{L}^+$ violating the interference condition:

Pruning rule 2 (States with non-interfering jobs pruning):

$$\forall x' = \vec{x}'(t) \in \mathcal{L}^+,$$

$$\text{IF } |\mathcal{Q}(x')| \leq m \quad \bigwedge \quad \exists i: c'_i = 1 \wedge d'_i \geq 1 \wedge h'_i = 0$$

$$\text{THEN } \mathcal{L}^+ := \mathcal{L}^+ \setminus \{x'\} \quad (31)$$

Before proving the correctness of the pruning rule above, we illustrate it with an example. Fig. 5 depicts two GEDF schedules A and B for 3 tasks and 2 cores, with interference slots being hatched for preempted jobs. In the Schedule A , the first job of τ_2 released at time $t=0$ and completed at $t=2$ does not cause interference on other jobs. Thus, according to the discussion above, we prune the states of the corresponding trajectory depicted above for $t \geq 2$ from the analysis; pruned states are hatched. Instead, we examine another legal trajectory corresponding to Schedule B , which does not contain that non-interfering job of τ_2 from the Schedule A . In turn, the states of the trajectory for the Schedule B are examined only for $t < 4$, as there is a non-interfering job of τ_3 completing at time $t=4$.

In Fig. 6 we depict the state transition graph of the same example of Fig. 1 pruned by (31). Observe that only 33 out of 91 states remain, and that the state with a missed deadline is not pruned.

The next theorem formalizes the discussion above, that the removal of non-interfering jobs from a GEDF schedule does not affect the schedulability.

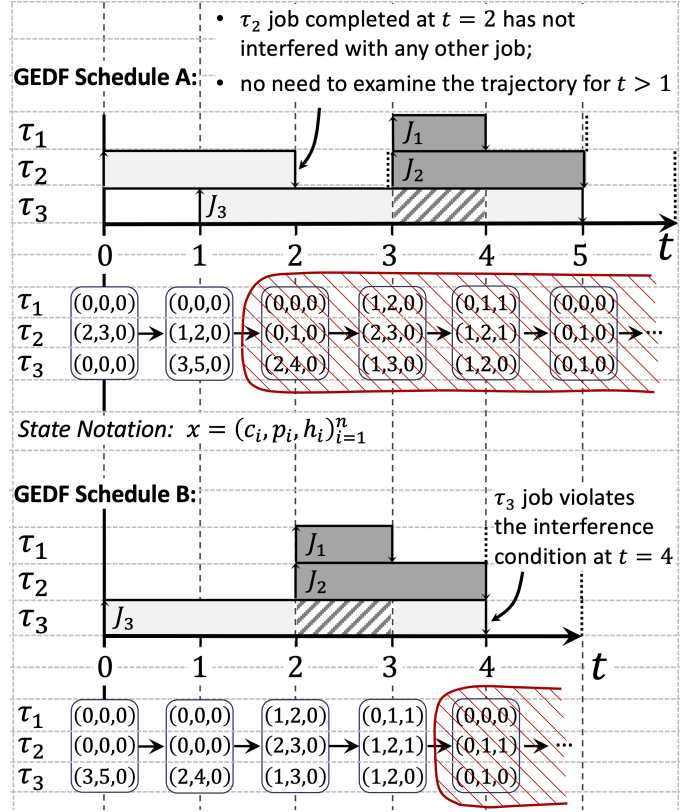


Fig. 5: Motivation example for a job interference condition: the removal of a non-interfering job does not affect the schedulability of other jobs. Task parameters are $\tau_1 = (1, 2)$, $\tau_2 = (2, 3)$, and $\tau_3 = (3, 5)$. State representation x is extended with the indicator of interference h_i

Theorem 2 (Job interference requirement for GEDF):

Let tasks be scheduled by GEDF. Let $\vec{x}$ and $\vec{x}'$ denote two trajectories with corresponding release sequences $r(t)$ and $r'(t)$, such that:

- In $\vec{x}'$, there is a job J_k of some task τ_k released at time t_r ($r'_k(t_r) = 1$), which does not interfere with other jobs (that is $\forall t \in [0, D_k), h_k(t_r + t) = 0$);
- The releases $r(t)$ of $\vec{x}$ are the same as $r'(t)$ except no release of J_k , that is

$$\forall i, \quad r_i(t) = \begin{cases} 0, & \text{if } t = t_r \wedge i = k \\ r'_i(t), & \text{otherwise} \end{cases} \quad (32)$$

Then, it holds that if some job, except J_k , misses a deadline in $\vec{x}'$, then it also misses its deadline in $\vec{x}$.

Observe that the theorem above does not apply to the job J_k , which is released in trajectory $\vec{x}'$ but not in $\vec{x}$. The reason is that, despite J_k causing no interference, other jobs can cause interference on J_k leading to its deadline miss. Thus, the trajectory $\vec{x}'$ should be examined until confirming the schedulability of J_k (see more details later).

The intuition behind the theorem above is that non-interfering jobs have no effect on the schedulability. Hence, any trajectory $\vec{x}'$, which has at least one non-interfering job J_k , can be discarded from the analysis, since it is as schedulable as the trajectory $\vec{x}$ without the job J_k , assuming that J_k does not miss its deadline. Thus, this theorem allows a significant

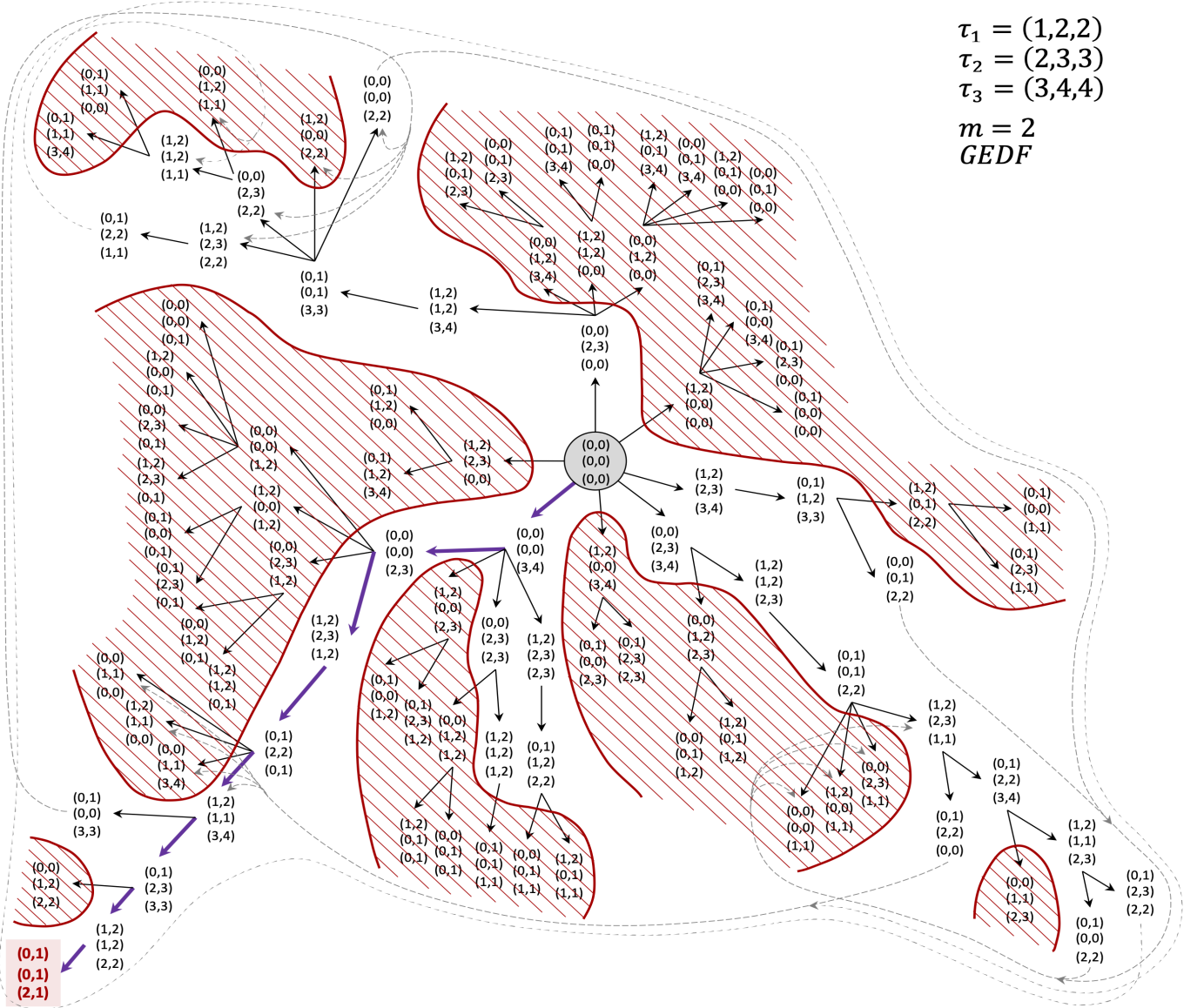


Fig. 6: State transition graph of Fig. 1 pruned by interference condition (31)

speed-up of the schedulability analysis by omitting the explicit exploration of equivalent trajectories.

The proof of Theorem 2 mimics the one of Theorem 1. First, we introduce an auxiliary lemma demonstrating that no job in trajectory $\vec{x}'$ can get less processing time than in $\vec{x}$. The proof of this lemma is conducted by contradiction by exploring the properties of the schedule function of (12). Then, we show that such an observation regarding the allocated processing time for trajectories $\vec{x}'$ and $\vec{x}$ results in Theorem 2.

Lemma 2: Let trajectories $\vec{x}$, $\vec{x}'$ and job J_k satisfy the condition of Theorem 2. Then, the following property holds:

$$\forall i \neq k, \forall t \notin [t_r, t_r + D_k), \\ q_i(\vec{x}', t) = 1 \Rightarrow s_i(\vec{x}, t) = s_i(\vec{x}', t), \quad (33)$$

and $s_i(\vec{x}, t)$, $q_i(\vec{x}, t)$ defined by (12).

To prove Lemma 2, we use the following properties of the

schedule function $s_i(\vec{x}, t)$ resulted from Definition 1:

$$s_i(\vec{x}, t) = 1 \Rightarrow \sum_{\tau_\ell \in \mathcal{T}^{\text{HP}}(\vec{x}(t), \tau_i)} q_\ell(\vec{x}, t) < m, \quad (34)$$

$$s_i(\vec{x}, t) = 0 \wedge q_i(\vec{x}, t) = 1 \Rightarrow \sum_{\tau_\ell \in \mathcal{T}^{\text{HP}}(\vec{x}(t), \tau_i)} q_\ell(\vec{x}, t) \geq m \quad (35)$$

According to the equations above, if a task τ_i is scheduled then the number of higher priority tasks is less than m . If instead a task τ_i is pending but not scheduled, then the number of higher priority tasks is at least m , to occupy all processors.

Proof: We prove the lemma by contradiction. Suppose that (33) is first violated by some task τ_{i^*} at t^* :

$$\exists i^*, t^* : q_{i^*}(\vec{x}', t^*) = 1 \wedge s_{i^*}(\vec{x}, t^*) \neq s_{i^*}(\vec{x}', t^*) \quad (36)$$

$$\forall i, t < t^* : q_i(\vec{x}', t) = 1 \Rightarrow s_i(\vec{x}, t) = s_i(\vec{x}', t). \quad (37)$$

As $s_i(\vec{x}, t)$, $s_i(\vec{x}', t)$ are boolean, (36) might hold in two cases:

$$\begin{cases} s_{i^*}(\vec{x}, t^*) = 1 \\ s_{i^*}(\vec{x}', t^*) = 0 \end{cases} \quad \text{or} \quad \begin{cases} s_{i^*}(\vec{x}, t^*) = 0 \\ s_{i^*}(\vec{x}', t^*) = 1 \end{cases} \quad (39)$$

(38)

We next show that both cases are infeasible.

Case 1: Suppose that (36) and the case of (38) hold. Substituting (34), (35) into (38), we get that

$$s_{i^*}(\vec{x}, t^*) = 1 \Rightarrow \sum_{\tau_\ell \in \mathcal{T}^{\text{HP}}(\vec{x}(t^*), \tau_{i^*})} q_\ell(\vec{x}, t^*) < m$$

$$\begin{cases} s_{i^*}(\vec{x}', t^*) = 0 \\ q_{i^*}(\vec{x}', t^*) = 1 \end{cases} \Rightarrow \sum_{\tau_\ell \in \mathcal{T}^{\text{HP}}(\vec{x}'(t^*), \tau_{i^*})} q_\ell(\vec{x}', t^*) \geq m.$$

By comparing the LHS of the inequalities above, it must exist a task $\tau_{i^{**}} \in \mathcal{T}^{\text{HP}}(\vec{x}'(t^*), \tau_{i^*})$ with $q_{i^{**}}(\vec{x}', t^*) = 1$ and satisfying one of two cases:

$$\begin{cases} q_{i^{**}}(\vec{x}, t^*) = 1 \\ \tau_{i^{**}} \notin \mathcal{T}^{\text{HP}}(\vec{x}(t^*), \tau_{i^*}) \end{cases} \quad \text{or} \quad q_{i^{**}}(\vec{x}, t^*) = 0 \quad (40)$$

In the first case, both τ_{i^*} and $\tau_{i^{**}}$ have pending jobs at states $\vec{x}(t^*)$ and $\vec{x}'(t^*)$, although the priority of the τ_{i^*} job is higher than of $\tau_{i^{**}}$ at state $\vec{x}(t^*)$ and vice versa at $\vec{x}'(t^*)$. From the theorem statement, the release times for τ_{i^*} and $\tau_{i^{**}}$ jobs in trajectories $\vec{x}$ and $\vec{x}'$ are the same, and so are their deadlines. Thus, we get a contradiction, as under the GEDF scheduler the priorities of τ_{i^*} and $\tau_{i^{**}}$ jobs should be the same at $\vec{x}(t^*)$ and $\vec{x}'(t^*)$.

In the second case, the task $\tau_{i^{**}}$ has the pending job J'_{**} , which is not finished yet at time t^* in trajectory $\vec{x}'$. Let t_r be its release. Then

$$\sum_{t=t_r}^{t^*-1} s_{i^{**}}(\vec{x}', t) < C_{i^{**}}. \quad (41)$$

Because of the “same releases” hypothesis of Th. 2, $\tau_{i^{**}}$ releases a job in trajectory $\vec{x}$ too, which we denote by J_{**} . Let t_f its finishing time, then the second case of (40) implies that J_{**} finishes by time t^* , that is $t_f \leq t^*$. Also we have

$$\sum_{t=t_r}^{t_f-1} s_{i^{**}}(\vec{x}, t) = C_{i^{**}}.$$

By comparing the equality above with (41), it must necessarily exist $\hat{t} \in [t_r, t_f]$ such that

$$q_{i^{**}}(\vec{x}', \hat{t}) = 1 \wedge s_{i^{**}}(\vec{x}', \hat{t}) = 0 \wedge s_{i^{**}}(\vec{x}, \hat{t}) = 1,$$

which contradicts (37) because $t_f \leq t^*$. Thus, the second case of (40) is also impossible.

Case 2: Suppose that (39) holds. Since τ_{i^*} is not scheduled at time t^* in trajectory $\vec{x}$, we consider two cases

- τ_{i^*} is not pending, that is $q_{i^*}(\vec{x}, t^*) = 0$
- τ_{i^*} is pending, that is $q_{i^*}(\vec{x}, t^*) = 1$

The first case with $q_{i^*}(\vec{x}, t^*) = 0$ and $q_{i^*}(\vec{x}', t^*) = 1$ from (36) is proved to be infeasible by analogy to case (40) by substituting τ_{i^*} instead of $\tau_{i^{**}}$.

In the second case, considering $q_{i^*}(\vec{x}, t^*) = 1$ and (38), we get that, for trajectories $\vec{x}$ and $\vec{x}'$ respectively:

$$\sum_{\tau_\ell \in \mathcal{T}^{\text{HP}}(\vec{x}(t^*), \tau_{i^*})} q_\ell(\vec{x}, t^*) \geq m \quad (42)$$

$$\sum_{\tau_\ell \in \mathcal{T}'^{\text{HP}}(\vec{x}'(t^*), \tau_{i^*})} q_\ell(\vec{x}', t^*) < m \quad (43)$$

From (42), for trajectory $\vec{x}$, there exists such a task $\tau_{i^{**}} \in \mathcal{T}^{\text{HP}}(\tau_{i^*}, t^*)$ having a job denoted by J_{**} , which interferes with τ_{i^*} at time t^* . Let t_r be the release time of J_{**} . According to (32), this $\tau_{i^{**}}$ must release a job at time t_r in trajectory $\vec{x}'$ as well, which we denote by J'_{**} . Then, considering (42) and (43), task $\tau_{i^{**}}$ must satisfy the conditions, for trajectories $\vec{x}$ and $\vec{x}'$ respectively:

$$\sum_{t=t_r}^{t^*-1} s_{i^{**}}(\vec{x}, t) < C_i$$

$$\sum_{t=t_r}^{t^*-1} s_{i^{**}}(\vec{x}', t) \geq C_i,$$

what gives contradiction to (37). Thus, both cases (38) and (39) are infeasible, and the lemma is proved. ■

We finally prove Theorem 2 by relying on Lemma 2.

Proof for Theorem 2:

Necessity: We first prove that the feasibility of $\vec{x}$ implies the feasibility of $\vec{x}'$. The proof is conducted by contradiction. Suppose that exist such $\vec{x}$ and $\vec{x}'$ satisfying the condition of the theorem, that $\vec{x}$ is feasible, but $\vec{x}'$ is infeasible. For $\vec{x}'$, let t_{dm} denote the earliest missed deadline and τ_i denote the task missing a deadline. Consequently, $t_{\text{dm}} - D_i$ is the release time of the job missing the deadline:

$$\sum_{t=t_{\text{dm}}-D_i}^{t_{\text{dm}}-1} s_i(\vec{x}', t) < C_i. \quad (44)$$

From the theorem statement, if τ_i releases a job at time $(t_{\text{dm}} - D_i)$ for $\vec{x}'$, then it also releases a job for $\vec{x}$, and, due to the assumption that $\vec{x}$ is feasible, we get

$$\sum_{t=t_{\text{dm}}-D_i}^{t_{\text{dm}}-1} s_i(\vec{x}, t) = C_i, \quad (45)$$

what contradicts to (44) due to Lemma 2.

Sufficiency: The sufficiency proof shows that feasibility of $\vec{x}'$ implies feasibility of $\vec{x}$. Such a proof is conducted by analogy to the necessity proof, by swapping functions $s_i(\vec{x}, t)$ and $s_i(\vec{x}', t)$ in Eq. (44)-(45).

The theorem follows. ■

We note that Pruning rule 2 appears to establish a connection with the critical instant for job releases - a long-standing open question in multiprocessor real-time scheduling theory [27]. We will investigate this observation in the future work, as well as the validity of this pruning rule to other schedulers.

VII. EVALUATION

Finally we report the evaluation results for the resulted exact schedulability test for GEDF¹. We run experiments upon the following hardware platform:

- Processor: 2.6 GHz 6-Core Intel Core i7 core;
- System memory: 16 GB 2667 MHz DDR4;

¹The C++ implementation of the test is available at <https://github.com/burmyakov/exact-sched-test-gedf>

Settings	Setup <i>A</i>	Setup <i>B</i>
Number of cores, m	2	4
Number of heavy tasks, n_{heavy}	arbitrary	5
Task set utilization, $U_{\mathcal{T}}$	1.5	3.3
Number of tasks, n	5	8
Range for task periods, P_i	[5, 25]	[5, 20]
Ratio of task periods, $P_{\max}/P_{\min}$	5	4
Ratio of task periods to deadlines	1	1

TABLE I: Key parameters: default values

- Operating system: MacOS Sonoma 14.0,

Default experiment settings are listed in Table I, grouped into Setups *A* and *B*, with the choice of parameters values justified below. For each experiment one parameter is varied, while others remain fixed to the default values of Table I. Task sets are generated by an ILP-based approach [24], as other approaches are inapplicable due to small task set parameters. We generate at least 50 task sets per a data point for most of the experiments. Due to the integer parameters and a limited n , some evaluation plots reported below are not smooth.

Setup A is to evaluate the performance gain of our test over the Bonifaci's test; for larger parameter values the Bonifaci's test becomes prohibitive. The results are reported in Figs. 7a, 8a and 7b, 8b. Thanks to the derived optimizations, the test speed-up is at least two orders of magnitude, which keeps growing with n , as well as the memory reduction is multiple orders of magnitude. The Bonifaci's test does not terminate for 5-6 tasks, due to exceeding the available 16 GB of system memory.

Setup B is to evaluate the performance of our test over larger task set parameter values. First, we choose the number of heavy² tasks exceeding the number of processors $n_{\text{heavy}} > m$, that is to justify the use of global instead of partitioned scheduling [28]. A task set utilization $U_{\mathcal{T}}$ is chosen sufficiently large to generate a required number n_{heavy} of heavy tasks. The key performance limitation of our test is its memory consumption, so that the choice of task parameters is constrained by the available 16 GB of system memory. The evaluation results are summarized in Figs. 9a-11a. Among other, we observe a significant impact of n_{heavy} on the test performance, as well as the schedulability ratio for generated task sets. A similar impact was observed by Burmyakov and Nikolic [28] for the Global Fixed Priority (GFP) scheduler. The reasons for such dependencies on n_{heavy} are to be studied later.

For the evaluation completeness, to motivate the need for an exact GEDF test, we evaluate the pessimism of the Bertogna's best-in-class sufficient test [25], [29]. The results are reported in Figures from 7c to 11c. We observe no schedulable task set detected by the sufficient test in case $n_{\text{heavy}} > m$, when the use of global scheduling makes sense, although in fact there are multiple schedulable task sets. Actually this fact supports the prior observation of Sun and Di Natale [30] and Biondi and Sun [31] regarding the pessimism of the available sufficient tests: any task set detected as schedulable by them is in fact schedulable by a trivial partition heuristic. Fig. 11c shows well the non-optimality of GEDF. In fact, the fraction of GEDF schedulable task sets drops as soon as the utilization exceeds a threshold (about 2.5 in the figure). To best of our knowledge,

this is the first evaluation of this type available in the literature of real-time scheduling.

Experiments of Fig. 10 shows a decrease of the runtime and memory, as the number n_{heavy} of tasks with utilization > 0.5 grows. Our explanation is that heavy tasks make a task set harder to be scheduled [1], [32], [33]. Then, our exact test stops as soon as a deadline miss is detected, without expanding the graph.

VIII. CONCLUSION

We proposed an exact schedulability test for the GEDF real-time scheduler by exploring the idea of a state-transition graph for modeling possible execution scenarios. To overcome significant analysis runtime and memory demand, we derived a set of optimizations called pruning rules for mitigating a state space explosion. The resulted test is several orders of magnitude faster and multiple orders of magnitude less memory consuming than other known best-in-class exact tests. Our test is implemented in C++ routines, and will be made publicly available.

Additionally, thanks to our improved exact test, we investigated various schedulability trends for synthetic real-time task sets. Some interesting trends we observed is the schedulability ratio increase with a number of tasks, but decrease with a heavy tasks number, if a task set utilization remains fixed. In fact, this confirms the prior observations of Burmyakov and Nikolic [28] regarding a strong impact of the number of heavy tasks on a task set schedulability and the efficiency of the global scheduling strategy in general.

Moreover, we briefly assessed the pessimism of the Bertogna's best-in-class sufficient schedulability test [25], [29] and confirmed its significance under certain setups. This is the first assessment of this type available in the real-time scheduling research literature.

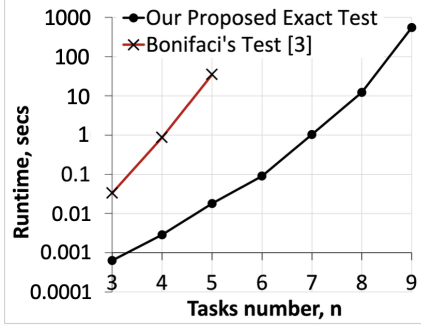
Pruning-driven state-transition graph analysis demonstrates good potential in deriving a more tractable exact schedulability test, but more efficient pruning optimizations are still to be derived. The problem is complicated by the dynamic nature of job priorities in case of the considered GEDF scheduler, unlike other ones such as GFP.

Moreover, we will investigate the validity of the Pruning rule 2 to other schedulers, as well as its connection to the critical instants for job releases — a long-standing open question in multiprocessor real-time scheduling theory [27].

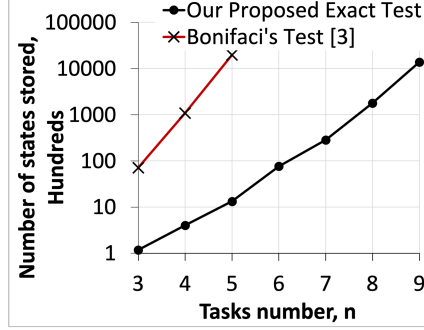
REFERENCES

- [1] J. Goossens, S. Funk, and S. Baruah, "Priority-driven scheduling of periodic task systems on multiprocessors," *Real-time systems*, vol. 25, pp. 187–205, 2003.
- [2] S. Baruah and K. Pruhs, "Open problems in real-time scheduling," *Journal of Scheduling*, vol. 13, no. 6, pp. 577–582, 2010.
- [3] V. Bonifaci and A. Marchetti-Spaccamela, "Feasibility analysis of sporadic real-time multiprocessor task systems," *Algorithmica*, vol. 63, pp. 763–780, 2012.
- [4] R. Alur and D. Dill, "Automata for modeling real-time systems," in *Automata, Languages and Programming: 17th International Colloquium Warwick University, England, July 16–20, 1990 Proceedings* 17. Springer, 1990, pp. 322–335.
- [5] G. Behrmann, K. G. Larsen, J. Pearson, C. Weise, and W. Yi, "Efficient timed reachability analysis using clock difference diagrams," in *Computer Aided Verification: 11th International Conference, CAV'99 Trento, Italy, July 6–10, 1999 Proceedings* 11. Springer, 1999, pp. 341–353.

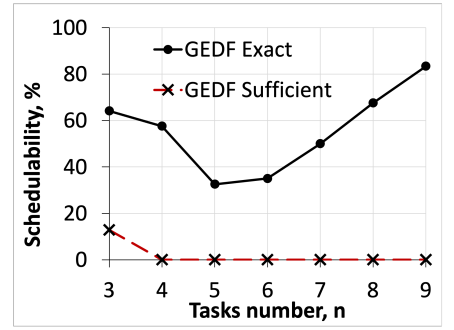
²A task τ_i is heavy when $C_i/D_i > 0.5$



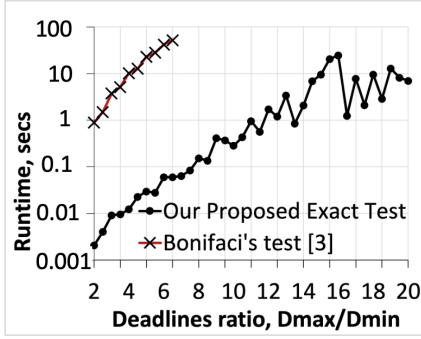
(a) Runtime



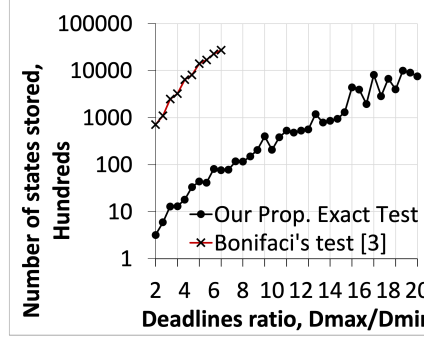
(b) States stored



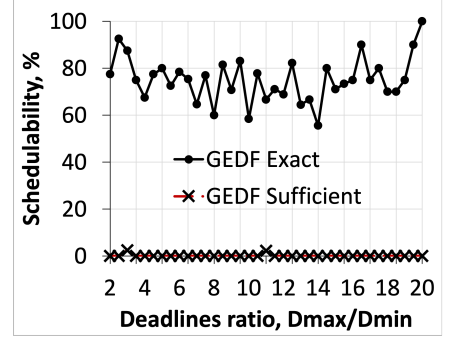
(c) Schedulability ratio

Fig. 7: Setup A: a varying number of tasks n 

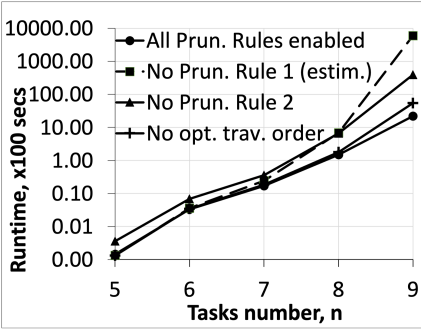
(a) Runtime



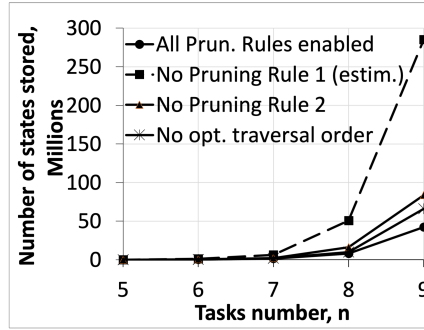
(b) States stored



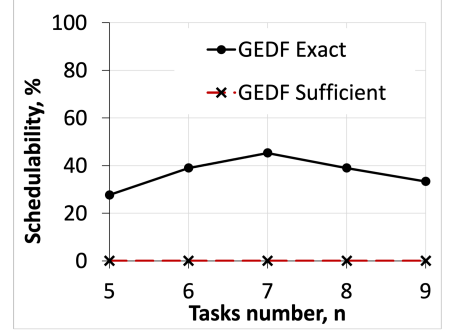
(c) Schedulability ratio

Fig. 8: Setup A: a varying ratio $D_{\max}/D_{\min}$ of task deadlines

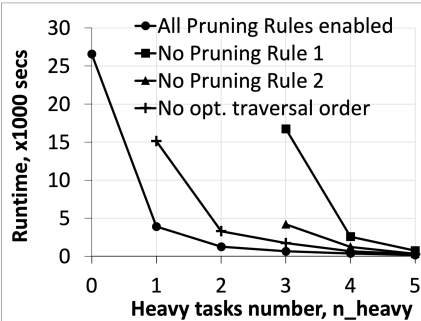
(a) Runtime



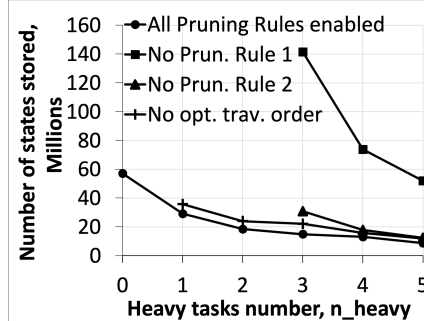
(b) States stored



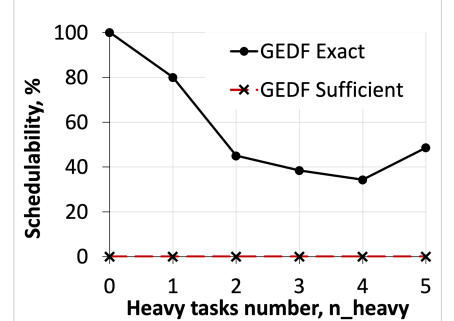
(c) Schedulability ratio

Fig. 9: Setup B: a varying number of tasks n 

(a) Runtime



(b) States stored



(c) Schedulability ratio

Fig. 10: Setup B: a varying number n_{heavy} of heavy tasks

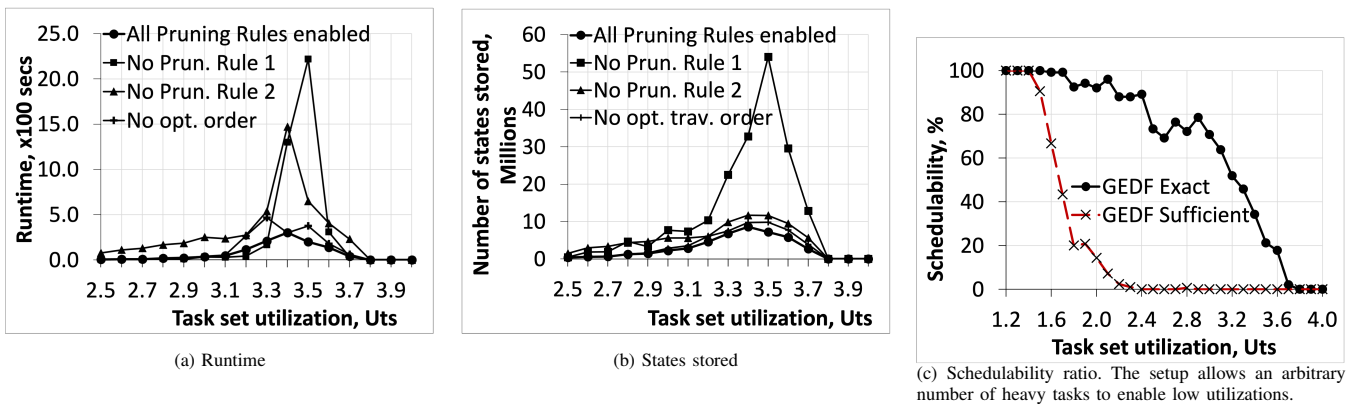


Fig. 11: Setup B: a varying task set utilization U_T

- [6] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International journal on software tools for technology transfer*, vol. 1, pp. 134–152, 1997.
- [7] G. Audemard, A. Cimatti, A. Kornilowicz, and R. Sebastiani, "Bounded model checking for timed systems," in *International Conference on Formal Techniques for Networked and Distributed Systems*. Springer, 2002, pp. 243–259.
- [8] E. Khamespanah, M. Sirjani, Z. S. Kaviani, R. Khosravi, and M.-J. Izadi, "Timed rebecca schedulability and deadlock freedom analysis using bounded floating time transition system," *Science of Computer Programming*, vol. 98, pp. 184–204, 2015.
- [9] T. A. Henzinger, X. Nicollin, J. Sifakis, and S. Yovine, "Symbolic model checking for real-time systems," *Information and computation*, vol. 111, no. 2, pp. 193–244, 1994.
- [10] A. Bouajjani, S. Tripakis, and S. Yovine, "On-the-fly symbolic model checking for real-time systems," in *Proceedings Real-Time Systems Symposium*. IEEE, 1997, pp. 25–34.
- [11] E. Bini and G. C. Buttazzo, "Schedulability analysis of periodic fixed priority systems," *IEEE Transactions on Computers*, vol. 53, no. 11, pp. 1462–1473, Nov. 2004.
- [12] A. Cimatti, L. Palopoli, and Y. Ramadian, "Symbolic computation of schedulability regions using parametric timed automata," in *2008 Real-Time Systems Symposium*. IEEE, 2008, pp. 80–89.
- [13] J. V. Pinxten, M. Geilen, and T. Basten, "Parametric scheduler characterization," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 1–25, 2019.
- [14] Y. Abdeddaïm and O. Maler, "Job-shop scheduling using timed automata?" in *Computer Aided Verification: 13th International Conference, CAV 2001 Paris, France, July 18–22, 2001 Proceedings 13*. Springer, 2001, pp. 478–492.
- [15] T. P. Baker and M. Cirinei, "Brute-force determination of multiprocessor schedulability for sets of sporadic hard-deadline tasks," in *International Conference On Principles Of Distributed Systems*. Springer, 2007, pp. 62–75.
- [16] G. Geeraerts, J. Goossens, and M. Lindström, "Multiprocessor schedulability of arbitrary-deadline sporadic tasks: complexity and antichain algorithm," *Real-time systems*, vol. 49, pp. 171–218, 2013.
- [17] J. Goossens, E. Grolleau, and L. Cucu-Grosjean, "Periodicity of real-time schedules for dependent periodic tasks on identical multiprocessor platforms," *Real-time systems*, vol. 52, pp. 808–832, 2016.
- [18] Y. Sun and G. Lipari, "A pre-order relation for exact schedulability test of sporadic tasks on multiprocessor global fixed-priority scheduling," *Real-Time Systems*, vol. 52, no. 3, pp. 323–355, 2016.
- [19] N. Guan, Z. Gu, Q. Deng, S. Gao, and G. Yu, "Exact schedulability analysis for static-priority global multiprocessor scheduling using model-checking," *Springer Lecture Notes in Computer Science*, vol. 4761, pp. 263–272, 2007.
- [20] J. Goossens and P. M. Yomsi, "Exact schedulability test for global-edf scheduling of periodic hard real-time tasks on identical multiprocessors," *arXiv preprint arXiv:1012.5929*, 2010.
- [21] M. Nasri and B. B. Brandenburg, "An exact and sustainable analysis of non-preemptive scheduling," in *2017 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2017, pp. 12–23.
- [22] S. Picard, A. Paolillo, G. Geeraerts, and J. Goossens, "Exact schedulability test for sporadic mixed-criticality real-time systems using antichains and oracles," in *The 32nd International Conference on Real-Time Networks and Systems (RTNS 2024)*, Porto, Portugal, Nov. 2024.
- [23] S. Ranjha, P. Gohari, G. Nelissen, and M. Nasri, "Partial-order reduction in reachability-based response-time analyses of limited-preemptive dag tasks," *Real-Time Systems*, vol. 59, no. 2, pp. 201–255, 2023.
- [24] A. Burmyakov, E. Bini, and C.-G. Lee, "Towards a tractable exact test for global multiprocessor fixed priority scheduling," *IEEE Transactions on Computers*, vol. 71, no. 11, pp. 2955–2967, 2022.
- [25] M. Bertogna and M. Cirinei, "Response-time analysis for globally scheduled symmetric multiprocessor platforms," in *28th IEEE International Real-Time Systems Symposium*. IEEE Computer Society, 2007.
- [26] M. Bertogna, M. Cirinei, and G. Lipari, "Schedulability analysis of global scheduling algorithms on multiprocessor platforms," *IEEE Transactions on Parallel and Distributed Systems*, vol. 20, no. 4, pp. 553–566, Apr. 2009.
- [27] R. Davis and A. Burns, "Priority assignment for global fixed priority preemptive scheduling in multiprocessor real-time systems," in *Proceedings of the 30th IEEE Real-Time Systems Symposium*, 2009.
- [28] A. Burmyakov and B. Nikolić, "An exact comparison of global, partitioned, and semi-partitioned fixed-priority real-time multiprocessor schedulers," *Journal of Systems Architecture*, vol. 121, p. 102313, 2021.
- [29] M. Bertogna, "Evaluation of existing schedulability tests for global edf," in *International Conference on Parallel Processing Workshops*. IEEE, 2009.
- [30] Y. Sun and M. D. Natale, "Assessing the pessimism of current multicore global fixed-priority schedulability analysis," University of Oxford, Tech. Rep., 2017.
- [31] A. Biondi and Y. Sun, "On the ineffectiveness of 1/m-based interference bounds in the analysis of global edf and fifo scheduling," *Real-Time Systems*, 2018.
- [32] S. K. Dhall and C. L. Liu, "On a real-time scheduling problem," *Operations research*, vol. 26, no. 1, pp. 127–140, 1978.
- [33] S. K. Baruah, "Optimal utilization bounds for the fixed-priority scheduling of periodic task systems on identical multiprocessors," *IEEE Transactions on Computers*, vol. 53, no. 6, pp. 781–784, 2004.



Artem Burmyakov is an Assistant Professor at the CSE Department of the Innopolis University specialised in real-time multiprocessor scheduling, static analysis of multithreaded programs, computer architecture and algorithms optimisation in general. Previously he was the post-doctoral researcher at the CSE Department of the Seull National University and the PhD student at the CISTER lab of the Instituto Politecnico do Porto, as well as a software researcher at the European Organisation for Nuclear Research (CERN). He obtained his Master's degree from the Moscow Engineering and Physics Institute. He published several research papers in the area of an exact real-time schedulability analysis and hierarchical scheduling, served as a reviewer of top-tier Computer Science journals and conferences.



Enrico Bini is Associate Professor at CS Department of University of Turin. Previously, he held positions at Scuola Superiore Sant'Anna and Lund University. In 2004, he completed the PhD on Real-Time Systems at Scuola Superiore Sant'Anna (recipient of the "Spitali Award" for best PhD thesis of the whole university). In January 2010 he also completed a Master degree in Mathematics with a thesis on optimal sampling for linear control systems.

He has published more than 100 papers (1 Test-of-Time award by the IEEE TCRTS, 4 best-paper awards) on real-time scheduling, operating systems, and optimization methods for real-time and control systems. He is Associate Editor of IEEE Transactions on Computers, ACM Computing Surveys, and Springer's Real-Time Systems journal. His service to the scientific community includes chairing 12 events (including Program Chair of RTSS'21) and the participation in 89 Technical Program Committees.